

RISC V Dual Core SoC Final Project Report

Synthesizable RTL Dual RV32I Integration Shared SRAM NoC Interconnect AXI4 Lite UART UVM ASIC Physical Design

Field	Entry
Student Name	Syed Muhammad Shahzim Maria Saleem Hamna Waseem
Submission Date	13/Sept/2026

Executive Summary

This report completes the final deliverable documentation for a compact dual-core RV32I system-on-chip. The supplied RTL integrates two synthesizable RV32I cores, private instruction memories, a shared SRAM subsystem, mailbox communication registers, a structured multi-core interconnect and an AXI4-Lite UART peripheral path. The design package also includes timing constraints, ASIC configuration files and a final chip-layout snapshot.

Verification is represented by a basic self-checking RTL testbench and a reusable UVM environment that drives the two core-side request ports directly. The UVM full-system test covers reset, shared SRAM transfer, simultaneous arbitration, address routing, mailbox transfer, back-to-back traffic, invalid-address behavior and UART/AXI-Lite completion. The test source reports a combined full-test summary with 14 pass checks and zero fail checks when the expected run completes.

The supplied ASIC package does not include the referenced LibreLane run directory or machine-readable timing, synthesis, physical-design and coverage report files. Where numerical signoff reports were absent, this document records the available design evidence and separates it from the previous-run summary stated in the provided README.

Final Deliverables Status

Deliverable	Status	Evidence in supplied package
Synthesizable RTL	Completed	asic/src contains synthesizable Verilog modules for the SoC, cores, interconnect, SRAM, mailbox and UART path.
Dual RV32I core integration	Completed	dual_core_soc instantiates core0 and core1 plus private instruction memories.
Instruction and shared SRAM integration	Completed	instruction_memory uses 32 words per core; shared_data_memory uses 64 32-bit words.
Structured multi-core interconnect	Completed	noc_interconnect handles requester selection, address decode, response routing and UART ownership locking.
Bus and peripheral interface	Completed	axi_lite_master_bridge and axi_uart_system connect memory-mapped requests to AXI4-Lite UART registers.
FPGA demonstration	RTL ready; board evidence not supplied	Top-level ports are clock, reset, UART TX and UART RX; no FPGA bitstream, board constraints or hardware screenshots were included.
Basic self-checking simulation	Completed	verify/tb_dual_core_soc.v checks shared memory results and prints pass/fail.
UVM verification environment	Completed	full_flow_uvm includes transaction, sequencer, driver, monitor, agent, environment, scoreboard and full-system test.
Timing constraints	Completed	impl.sdc and signoff.sdc define clock, I/O delays, reset false path, clock uncertainty, transition, fanout, capacitance and derating.
Synthesis and physical-design reports	Partially evidenced	config.yaml and README summarize the OpenLane/LibreLane setup and previous GDS result; report files were not present in the archive.

Final layout screenshots	Completed	The supplied chip snapshot is included in this report.
Brief architecture document	Completed	This report contains the architecture description, memory map, verification environment and implementation evidence.

System Architecture

The top-level design is organized around two non-pipelined RV32I cores. Each core fetches instructions from an independent instruction memory and issues load/store activity to a shared subsystem. The shared subsystem contains SRAM, mailbox registers and an AXI4-Lite UART path. A central interconnect arbitrates the two core request streams and returns read data and ready status to the selected core.

Block	Main role
dual_core_soc	Top-level integration of two RV32I cores, two instruction memories and the shared mailbox system.
rv32i_core	Implements RV32I execution, register file writeback, branch/jump control and memory wait handling.
instruction_memory	Private per-core instruction storage initialized from firmware_core0.hex and firmware_core1.hex.
mailbox_system	Wraps interconnect, shared SRAM, mailbox register and AXI4-Lite UART subsystem.
noc_interconnect	Collects two core request streams, arbitrates, decodes address regions and routes responses.
shared_data_memory	64-word shared 32-bit SRAM with registered request/ready behavior.
mailbox_reg	Message and valid/status registers for core-to-core communication.
axi_lite_master_bridge	Converts simple read/write requests into AXI4-Lite transactions.
axi_uart_system	Combines AXI UART slave, transmit path, receive path and baud generator.

Memory Map

The interconnect decodes shared accesses into SRAM, mailbox and UART regions. Unmapped accesses complete safely with zero read data.

Address or range	Resource	Behavior
0x00000000 to 0x000000FF	Shared SRAM physical storage	64 words of 32-bit memory; region decoded by high address bits.
0x00001000	Mailbox message	Write stores message and sets valid; read returns message and clears valid.
0x00001004	Mailbox status	Read returns valid in bit 0.
0x00002000	UART TX data	Write launches UART transmit activity through AXI4-Lite path.
0x00002004	UART status	Read returns TX busy, RX valid, parity error and frame error status bits.

0x00002008	UART RX data	Read returns received byte data.
Other addresses	Unmapped	Interconnect returns ready with zero read data.

Dual RV32I Core Integration

The two cores share the same RV32I implementation and are instantiated as independent processors. Each processor contains a program counter, register file, ALU, immediate generator and control unit. Register x0 is held at zero by the register-file implementation, and writeback supports ALU results, memory read data, PC plus four and immediate values.

The core interface exposes memory read, memory write, byte write strobe, address, write data, read data and ready signals. When a load/store instruction is active and the shared subsystem has not asserted ready, the core holds its current execution state. This wait behavior allows the interconnect and AXI4-Lite peripheral path to take multiple cycles without corrupting program flow.

Structured Multi Core Interconnect

The interconnect behaves as the structured communication fabric for the two-core system. It performs request collection, arbitration, address decode, target enable generation and response routing. When both cores request service at the same time, the design uses a stored grant state to alternate service under contention. UART transactions are protected by an ownership lock so that a multi-cycle AXI4-Lite access remains associated with the core that started it.

- SRAM requests are passed to shared_data_memory and complete when mem_ready returns.
- Mailbox requests are served directly through mailbox_reg and complete in the selected interconnect cycle.
- UART requests are routed to the AXI4-Lite bridge and complete only when uart_ready returns.
- Invalid or unmapped requests are completed without hanging the requesting core.

Bus and Peripheral Interface

The UART peripheral is attached through a dedicated AXI4-Lite master bridge. The bridge drives AW, W, B, AR and R channel behavior from the SoC request interface. A turnaround state prevents a still-asserted core request from being interpreted as a duplicate peripheral operation before the processor advances.

UART status bit	Meaning
3	Frame error
2	Parity error
1	RX valid
0	TX busy

Basic Self Checking Simulation

The RTL simulation testbench tb_dual_core_soc instantiates the full top level, generates a 50 MHz clock with a 20 ns period, applies reset, dumps a VCD waveform and then checks the internal shared-memory results. The testbench fails immediately if either expected value is wrong and prints a pass message when both cores complete their programs.

Check	Expected value	Purpose
shared_memory[0]	15	Confirms Core0 firmware executed and wrote its result.
shared_memory[1]	3	Confirms Core1 firmware executed and wrote its result.
PASS message	both cores completed their programs	Confirms the testbench reached the success path.

UVM Verification Environment

The UVM environment targets the shared subsystem below the processors and emulates the two core-side requesters. This gives precise control over concurrent traffic while still exercising the interconnect, shared SRAM, mailbox and UART path.

UVM file or class	Role
core_transaction_uvm	Transaction fields for read, write, address, write data, read data and ready.
core_sequence_uvm	Creates directed core-side transactions and captures completion data.
core_driver_uvm	Drives the request interface and waits for ready.
core_monitor_uvm	Publishes completed transactions through analysis ports.
core_agent_uvm	Packages sequencer, driver and monitor for each core requester.
soc_scoreboard_uvm	Reference model for shared SRAM writes and read comparisons.
soc_env_uvm	Instantiates the two agents and shared scoreboard.
full_system_test_uvm	Runs the integrated reset, SRAM, arbitration, routing, mailbox, invalid address and UART checks.

Scenario	Expected result
Reset / SRAM initial value	Read returns zero after reset.
Shared SRAM Core0 to Core1	Core1 reads data written by Core0.
Concurrent arbitration	Both requests complete in serialized order.
SRAM routing	SRAM address maps to shared memory.
Mailbox communication	Message, valid flag and clear-on-read behavior match specification.
Back-to-back accesses	Sequential writes and reads return all expected values.
Invalid address	Unmapped read returns zero and completes.
UART AXI-Lite path	TX write and status read complete through the UART route.

The full-system test source maintains pass_count and fail_count and reports FULL_TEST_SUMMARY at the end of the run. The expected completed run summary is PASS=14 and FAIL=0. The scoreboard summary format is

WRITES, READS, MATCHES and ERRORS; the earlier report transcript records WRITES=7, READS=8, MATCHES=7 and ERRORS=0.

Timing Constraints and ASIC Configuration

The design includes implementation and signoff SDC files plus a Sky130 ASIC configuration. Both SDC files define the main clock, I/O delays, reset false path and design-rule constraints used by physical implementation.

Constraint or setting	Value
Top design	dual_core_soc
PDK	sky130A
Clock port	clk
Clock period	20.0 ns
I/O delay constraint	20 percent of clock period
Clock uncertainty	0.25 ns
Clock transition	0.15 ns
Maximum fanout	32
Maximum transition	0.75 ns
Maximum capacitance	0.2 pF
Output load	33.442 fF
Die area	1000 um by 1000 um
Core area	30 um to 970 um in both X and Y
Placement target density	0.50
PDN layers	met5 horizontal, met4 vertical

Synthesis and Physical Design Evidence

The ASIC configuration is structured for a LibreLane/OpenLane-style flow using the Sky130 process and the supplied synthesizable RTL. The README states that a previous run reached GDS and completed DRC, LVS, antenna, XOR and hold checks with zero violations. It also states that the main remaining issue in that previous run was worst setup slack of about -3.47 ns at the slow corner, plus slew and fanout violations.

The updated configuration responds to that timing concern by keeping a 20 ns clock for the optimized run, enabling the supplied SDC files, using a delay-oriented synthesis strategy, increasing maximum fanout to 32, reducing PDN pitch and using a 1000 um by 1000 um die. Because the run directory and detailed report files were absent from the provided archive, this report does not invent final WNS, TNS, utilization, cell area or routed power numbers.

Report item	Documented status
Synthesis setup	Present in config.yaml and RTL file list.
Implementation SDC	Present as src/impl.sdc.

Signoff SDC	Present as src/signoff.sdc.
Pin order	Present as pin_order.cfg with clk/reset and UART pins.
GDS/DRC/LVS summary	Summarized in README, detailed run artifacts not included.
Final timing report	Not included as a report file.
Final area report	Not included as a report file.
Final power report	Not included as a report file.

Final Layout Screenshot

The supplied chip snapshot is included below as the final layout visual evidence. It shows the routed design inside the die boundary with visible power grid and routed signal metal across the core area.

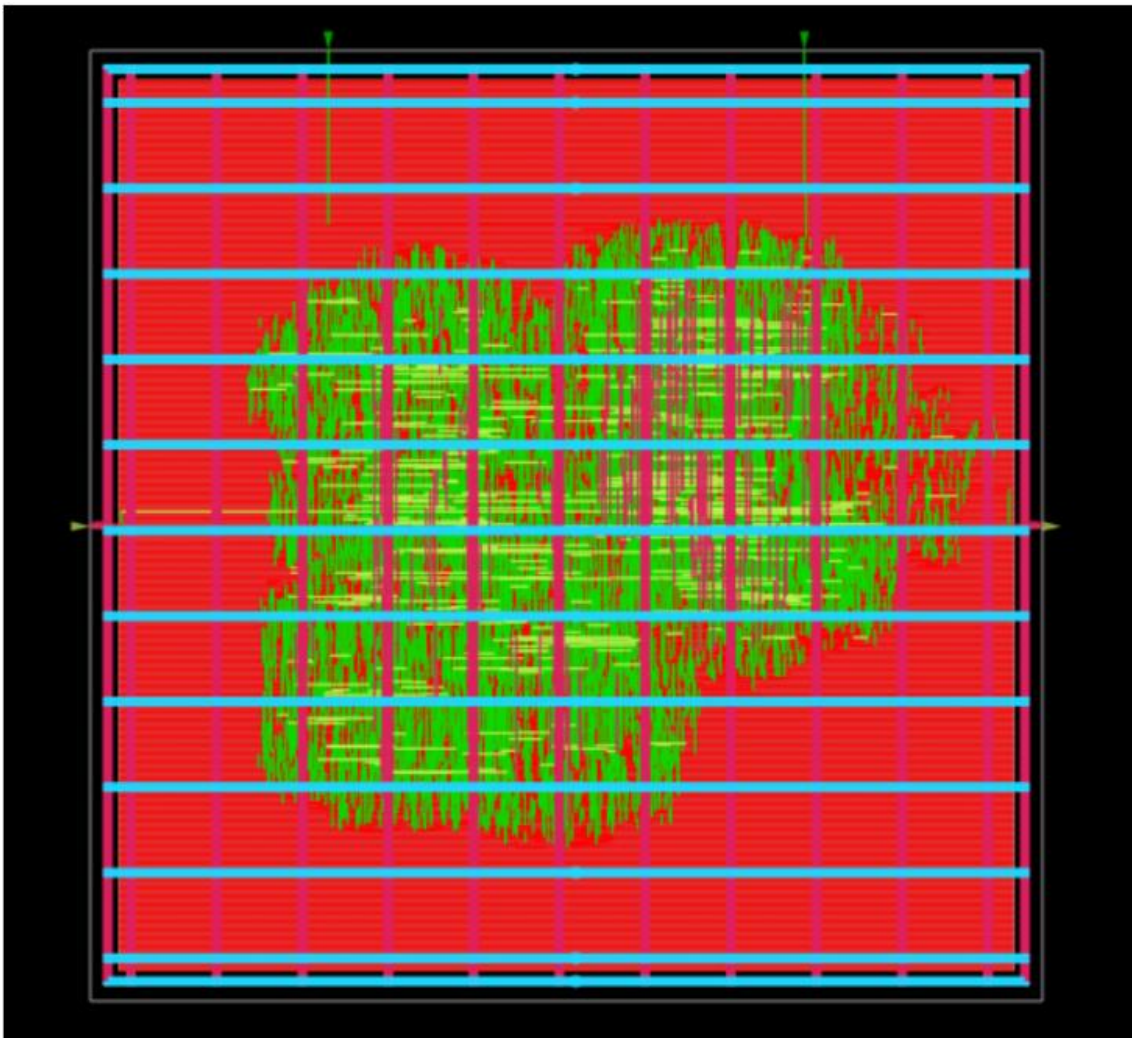


Figure 1 Final chip layout snapshot supplied with the project

Conclusion

The supplied project implements the requested dual-core RV32I SoC architecture in synthesizable RTL and documents a coherent shared-resource subsystem. The two cores execute from private instruction memories and communicate with shared SRAM, mailbox registers and a UART peripheral through a structured interconnect. The verification material includes a basic self-checking full-SoC simulation and a reusable UVM testbench that exercises the central integration risks: arbitration, routing, mailbox behavior, invalid addresses and UART/AXI-Lite completion.

The final report also captures the ASIC timing constraints and physical-design setup that were present in the archive, plus the provided final layout screenshot. The only missing machine-readable evidence in the supplied files is the detailed physical-design run directory, numerical synthesis/timing/power reports, FPGA hardware proof and formal coverage report output.